

XGBoost Coffee Quality Prediction

Comprehensive Analysis with Multiple Modeling Approaches

2025-12-28

Abstract

This report analyzes the Coffee Quality Institute's Arabica coffee dataset using XGBoost regression to predict Cupper Points (expert quality scores). The analysis compares multiple feature sets (Sensory, Extended, Full), evaluates baseline models, and documents both successful approaches and those found less effective (Lambda/Min_Child_Weight with XGBoost providing modest but statistically significant improvements over Linear Regression. regularization, Bayesian Optimization). Key finding: sensory features dominate prediction,

Contents

1	Introduction	4
1.1	Configuration	4
2	Data Loading and Preprocessing	5
2.1	Load Data	5
2.2	Clean Categorical Variables	5
2.3	Clean Numeric Variables	6
2.4	Remove Leakage Columns	8
3	Exploratory Data Analysis	9
3.1	Feature Descriptions	9
3.2	Target Variable Analysis	9
3.3	Correlation Analysis	9
4	Feature Engineering	13
4.1	Feature Set Definitions	13
4.2	Prepare Clean Dataset	13
5	Train/Test Split	14
6	Evaluation Metrics	14
7	Baseline Models	16
7.1	Mean Predictor	16
7.2	Linear Regression	16

8	XGBoost Hyperparameter Optimization	18
8.1	Grid Search Setup	18
8.2	Run HPO	18
8.3	Optimal Hyperparameters	18
9	Train Final Models	20
10	Learning Curves	20
11	Test Set Results	21
11.1	Standard Metrics	21
11.2	Enhanced Metrics	21
11.3	Performance Visualization	22
11.4	Improvement Summary	22
12	Predicted vs Actual Plots	23
13	Residual Analysis	23
14	Feature Importance	24
15	10-Fold Cross-Validation	25
15.1	Run CV	25
15.2	CV Summary	25
15.3	CV Visualization	25
16	Statistical Significance	26
16.1	Paired t-Tests	26
16.2	Effect Sizes	26
17	Approaches Found Less Effective	27
17.1	Lambda and Min_Child_Weight Regularization	27
17.2	Bayesian Optimization vs Grid Search	27
18	Final Summary	29
18.1	Key Findings	29
19	Explainable AI: PFI and PDP Analysis	30
19.1	Setup: Load DALEX and Create Explainer	30
19.2	Permutation Feature Importance (PFI)	31
19.3	Partial Dependence Plots (PDP)	34
19.4	Microscopic Explanations (Individual Predictions)	39
19.5	Summary: XAI Insights for XGBoost Full Model	44

20 Appendix	47
20.1 Appendix A: System Information	47

1 Introduction

This comprehensive report analyzes coffee quality prediction using XGBoost regression. The analysis incorporates multiple modeling approaches tested across several experiments.

Successful Approaches:

- Full model comparison (Sensory, Extended, Full feature sets vs baselines)
- Enhanced evaluation metrics (MAE, MAPE, Adjusted R^2 , Max Error)
- Learning Curves for training dynamics visualization
- Exploratory Data Analysis with Correlation Analysis
- Predicted vs Actual diagnostic plots

Approaches Tested but Found Less Effective:

- Lambda (L2) and Min_Child_Weight regularization tuning
- Bayesian Optimization vs Grid Search comparison

1.1 Configuration

Table 1: Analysis Configuration

Parameter	Value
Random Seed	42
Cross-Validation Folds	10
Train/Test Split	80/20
Early Stopping Rounds	50
Max Boosting Rounds	2000

2 Data Loading and Preprocessing

2.1 Load Data

```
coffee_raw <- read.csv("arabica_data_cleaned.csv")
cat("Raw data:", nrow(coffee_raw), "rows,", ncol(coffee_raw), "columns\n")
```

```
## Raw data: 1311 rows, 44 columns
```

2.2 Clean Categorical Variables

```
# Processing Method
coffee_raw$Processing.Method <- ifelse(
  is.na(coffee_raw$Processing.Method) | coffee_raw$Processing.Method == "",
  "Unknown", coffee_raw$Processing.Method
)
coffee_raw$Processing.Method <- factor(coffee_raw$Processing.Method)

# Variety (Top 4 + Other)
variety_counts <- sort(table(coffee_raw$Variety), decreasing = TRUE)
top_4_varieties <- names(variety_counts)[1:4]
coffee_raw$Variety <- ifelse(
  is.na(coffee_raw$Variety) | coffee_raw$Variety == "" |
    !(coffee_raw$Variety %in% top_4_varieties),
  "Other", coffee_raw$Variety
)
coffee_raw$Variety <- factor(coffee_raw$Variety)

# Country (Top 10 + Other)
country_counts <- sort(table(coffee_raw$Country.of.Origin), decreasing = TRUE)
top_10_countries <- names(country_counts)[1:10]
coffee_raw$Country.of.Origin <- ifelse(
  is.na(coffee_raw$Country.of.Origin) |
    !(coffee_raw$Country.of.Origin %in% top_10_countries),
  "Other", coffee_raw$Country.of.Origin
)
coffee_raw$Country.of.Origin <- factor(coffee_raw$Country.of.Origin)

# Color
coffee_raw$Color <- ifelse(
  is.na(coffee_raw$Color) | coffee_raw$Color == "",
  "Unknown", coffee_raw$Color
)
coffee_raw$Color <- factor(coffee_raw$Color)

# In-Country Partner (Top 10 + Other)
partner_counts <- sort(table(coffee_raw$In.Country.Partner), decreasing = TRUE)
top_10_partners <- names(partner_counts)[1:10]
coffee_raw$In.Country.Partner <- ifelse(
  is.na(coffee_raw$In.Country.Partner) |
```

```

    !(coffee_raw$In.Country.Partner %in% top_10_partners),
    "Other", coffee_raw$In.Country.Partner
)
coffee_raw$In.Country.Partner <- factor(coffee_raw$In.Country.Partner)

# Region (Top 15 + Other)
region_counts <- sort(table(coffee_raw$Region), decreasing = TRUE)
top_15_regions <- names(region_counts)[1:15]
coffee_raw$Region <- ifelse(
  is.na(coffee_raw$Region) | coffee_raw$Region == "" |
  !(coffee_raw$Region %in% top_15_regions),
  "Other", coffee_raw$Region
)
coffee_raw$Region <- factor(coffee_raw$Region)

cat("Categorical variables processed\n")

```

```
## Categorical variables processed
```

```
cat(" Processing.Method:", nlevels(coffee_raw$Processing.Method), "levels\n")
```

```
## Processing.Method: 6 levels
```

```
cat(" Variety:", nlevels(coffee_raw$Variety), "levels\n")
```

```
## Variety: 4 levels
```

```
cat(" Country:", nlevels(coffee_raw$Country.of.Origin), "levels\n")
```

```
## Country: 11 levels
```

```
cat(" Color:", nlevels(coffee_raw$Color), "levels\n")
```

```
## Color: 5 levels
```

2.3 Clean Numeric Variables

```

# Altitude (Cap at 3000m and Impute)
coffee_raw$altitude_mean_meters <- ifelse(
  coffee_raw$altitude_mean_meters > 3000 & !is.na(coffee_raw$altitude_mean_meters),
  3000, coffee_raw$altitude_mean_meters
)

region_medians <- coffee_raw %>%
  filter(!is.na(altitude_mean_meters)) %>%
  group_by(Region) %>%
  summarise(region_alt = median(altitude_mean_meters), .groups = "drop")

```

```

global_median_alt <- median(coffee_raw$altitude_mean_meters, na.rm = TRUE)

coffee_raw <- coffee_raw %>%
  left_join(region_medians, by = "Region") %>%
  mutate(altitude_mean_meters = ifelse(
    is.na(altitude_mean_meters),
    ifelse(is.na(region_alt), global_median_alt, region_alt),
    altitude_mean_meters
  )) %>%
  select(-region_alt)

# Harvest Year Parsing
parse_harvest_year <- function(val) {
  if (is.na(val) || val == "") return(NA)
  match <- str_extract(as.character(val), "(20\\d{2}|19\\d{2})")
  if (!is.na(match)) return(as.integer(match))
  return(NA)
}

coffee_raw$Harvest.Year.Parsed <- sapply(coffee_raw$Harvest.Year, parse_harvest_year)
median_year <- median(coffee_raw$Harvest.Year.Parsed, na.rm = TRUE)
coffee_raw$Harvest.Year.Parsed <- ifelse(
  is.na(coffee_raw$Harvest.Year.Parsed), median_year, coffee_raw$Harvest.Year.Parsed
)

# Bag Weight Standardization
parse_bag_weight_kg <- function(val) {
  if (is.na(val)) return(NA)
  val <- tolower(as.character(val))
  num <- as.numeric(str_extract(val, "[\\d.]+"))
  if (is.na(num)) return(NA)
  if (grepl("lb", val)) return(num * 0.453592)
  if (grepl("kg", val)) return(num)
  return(ifelse(num > 10, num, NA))
}

coffee_raw$Bag.Weight.KG <- sapply(coffee_raw$Bag.Weight, parse_bag_weight_kg)
median_weight <- median(coffee_raw$Bag.Weight.KG, na.rm = TRUE)
coffee_raw$Bag.Weight.KG <- ifelse(
  is.na(coffee_raw$Bag.Weight.KG), median_weight, coffee_raw$Bag.Weight.KG
)

cat("Numeric variables processed\n")

```

```
## Numeric variables processed
```

```
cat("  Altitude missing:", sum(is.na(coffee_raw$altitude_mean_meters)), "\n")
```

```
##   Altitude missing: 0
```

```
cat("  Harvest.Year range:", min(coffee_raw$Harvest.Year.Parsed), "-",
    max(coffee_raw$Harvest.Year.Parsed), "\n")
```

```
## Harvest.Year range: 2009 - 2018
```

2.4 Remove Leakage Columns

```
coffee_raw$Total.Cup.Points <- NULL
coffee_raw$Altitude <- NULL
coffee_raw$Species <- NULL
coffee_raw$Harvest.Year <- NULL
coffee_raw$Bag.Weight <- NULL

cat("Leakage columns removed: Total.Cup.Points, Altitude, Species, Harvest.Year, Bag.Weight\n")
```

```
## Leakage columns removed: Total.Cup.Points, Altitude, Species, Harvest.Year, Bag.Weight
```

3 Exploratory Data Analysis

3.1 Feature Descriptions

Table 2: Sensory Feature Descriptions

Feature	Description	Scale
Aroma	Smell/fragrance of the coffee	0-10
Flavor	Overall taste profile	0-10
Aftertaste	Lingering taste after swallowing	0-10
Acidity	Bright, tangy quality	0-10
Body	Weight/thickness on the tongue	0-10
Balance	How well flavors harmonize	0-10
Uniformity	Consistency across multiple cups	0-10
Clean.Cup	Absence of off-flavors/defects	0-10
Sweetness	Natural sweetness perception	0-10
Moisture	Moisture content of beans (%)	%
Category.One.Defects	Count of major defects	Count
Quakers	Unripe/underdeveloped beans count	Count

3.2 Target Variable Analysis

Table 3: Target Variable (Cupper.Points) Summary

Statistic	Value
Mean	7.4979
Std Dev	0.4746
Median	7.5000
Min	0.0000
Max	10.0000
IQR	0.5000

3.3 Correlation Analysis

Table 4: Feature Correlations with Cupper.Points

Feature	Correlation
Flavor	0.7971
Aftertaste	0.7878
Balance	0.7415
Acidity	0.7004
Aroma	0.6934
Body	0.6711
Uniformity	0.3585
Clean.Cup	0.3570
Sweetness	0.3001
Quakers	0.0119

Feature	Correlation
Category.One.Defects	-0.0527
Moisture	-0.1792

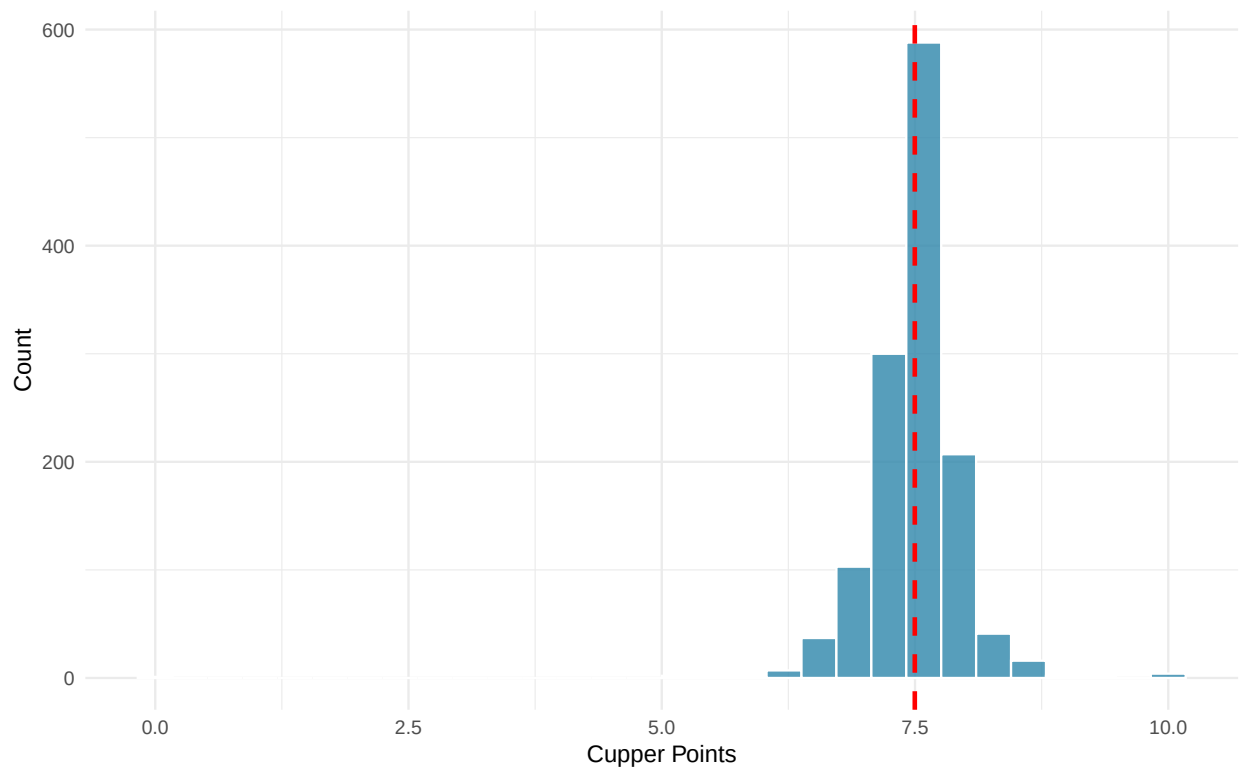


Figure 1: Distribution of Cupper Points. Red dashed line indicates mean.

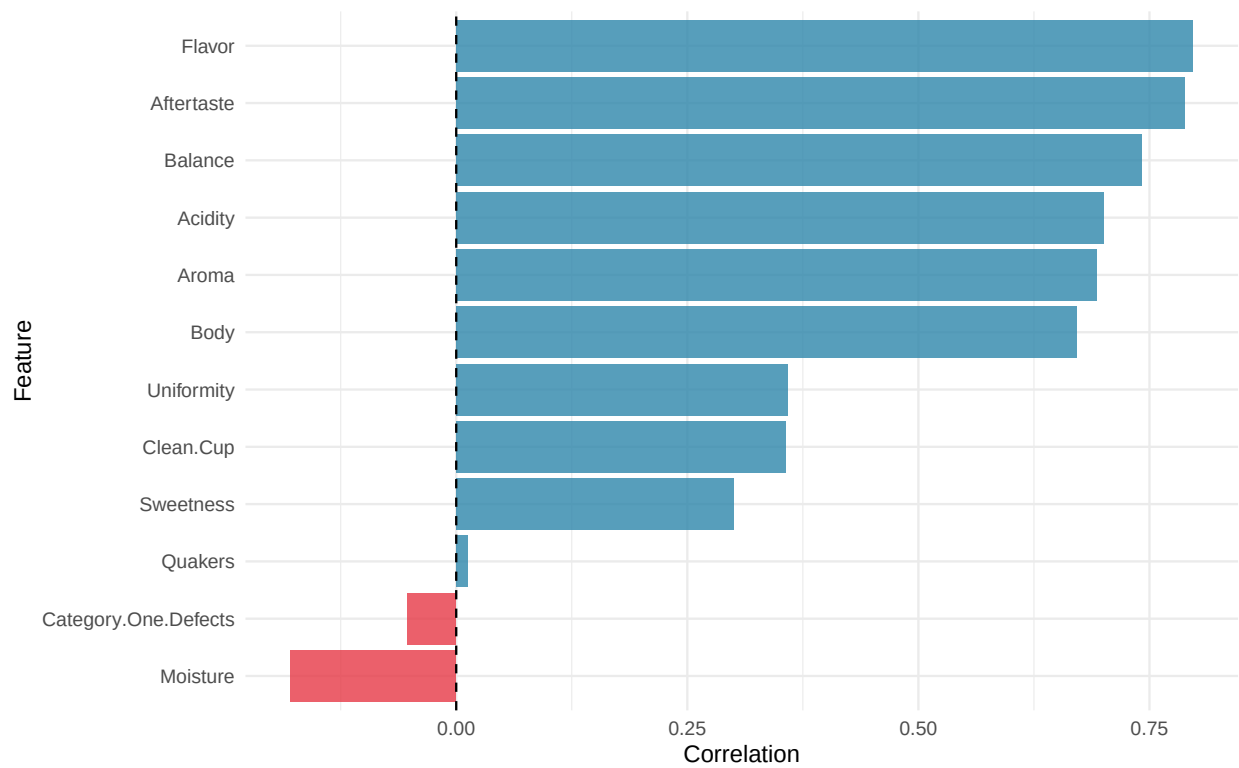


Figure 2: Feature correlations with Cupper Points.

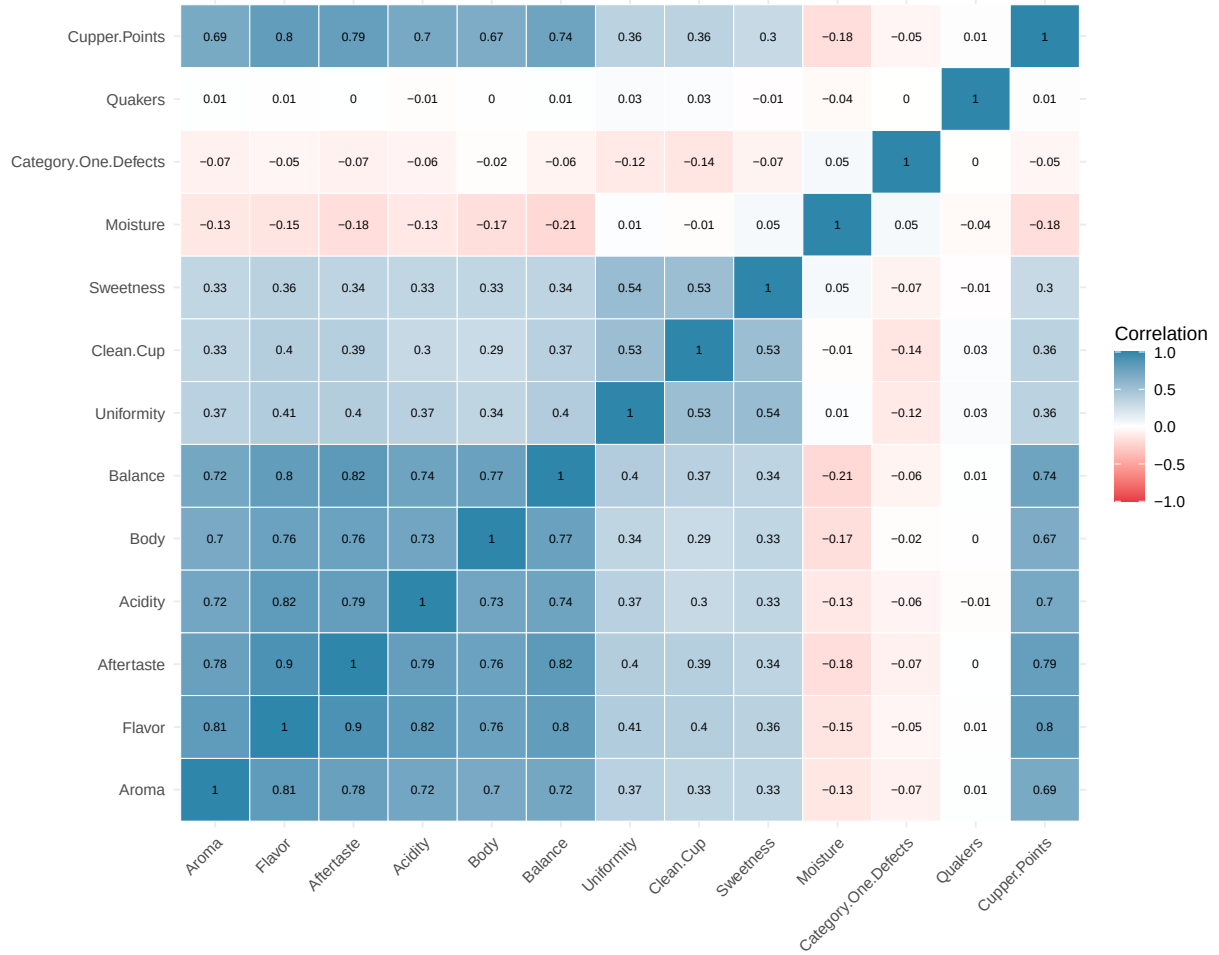


Figure 3: Correlation heatmap of sensory features and target variable.

4 Feature Engineering

4.1 Feature Set Definitions

```
sensory_cols <- c("Aroma", "Flavor", "Aftertaste", "Acidity", "Body",  
                  "Balance", "Uniformity", "Clean.Cup", "Sweetness",  
                  "Moisture", "Category.One.Defects", "Quakers")  
  
extended_cols <- c(sensory_cols,  
                   "Processing.Method", "Variety",  
                   "altitude_mean_meters", "Category.Two.Defects")  
  
full_cols <- c(extended_cols,  
               "Country.of.Origin", "Color", "Region",  
               "Harvest.Year.Parsed", "Bag.Weight.KG",  
               "Number.of.Bags", "In.Country.Partner")
```

Table 5: Feature Set Definitions

Feature.Set	N.Features	Description
Sensory	12	Core sensory quality indicators only
Extended	16	Sensory + key metadata (processing, variety, altitude)
Full	23	All available features including origin and logistics

4.2 Prepare Clean Dataset

```
coffee_clean <- coffee_raw[, c(full_cols, "Cupper.Points")]  
coffee_clean <- na.omit(coffee_clean)  
cat("Clean dataset:", nrow(coffee_clean), "rows\n")
```

```
## Clean dataset: 1310 rows
```

5 Train/Test Split

```
set.seed(SEED)

n <- nrow(coffee_clean)
train_idx <- sample(1:n, floor(0.8 * n))
test_idx <- setdiff(1:n, train_idx)

y <- coffee_clean$Cupper.Points
y_train <- y[train_idx]
y_test <- y[test_idx]

# Sensory features
X_sensory <- as.matrix(coffee_clean[, sensory_cols])
X_sensory_train <- X_sensory[train_idx, ]
X_sensory_test <- X_sensory[test_idx, ]

# Extended features (one-hot encoded)
X_extended <- model.matrix(~ . - 1, data = coffee_clean[, extended_cols])
X_extended_train <- X_extended[train_idx, ]
X_extended_test <- X_extended[test_idx, ]

# Full features (one-hot encoded)
X_full <- model.matrix(~ . - 1, data = coffee_clean[, full_cols])
X_full_train <- X_full[train_idx, ]
X_full_test <- X_full[test_idx, ]

train_df <- coffee_clean[train_idx, ]
test_df <- coffee_clean[test_idx, ]

# CV folds
set.seed(SEED + 42)
fold_ids <- sample(rep(1:K_FOLDS, length.out = n))
```

Table 6: Train/Test Split Summary

Set	Observations	Percentage
Training	1048	80%
Test	262	20%
Total	1310	100%

6 Evaluation Metrics

```
get_metrics <- function(pred, obs) {
  residuals <- obs - pred
  rmse <- sqrt(mean(residuals^2))
  ss_res <- sum(residuals^2)
  ss_tot <- sum((obs - mean(obs))^2)
```

```

r2 <- 1 - (ss_res / ss_tot)
mae <- mean(abs(residuals))
return(list(RMSE = rmse, R2 = r2, MAE = mae))
}

get_enhanced_metrics <- function(pred, obs, n_features = 12) {
  residuals <- obs - pred
  n <- length(obs)

  rmse <- sqrt(mean(residuals^2))
  mae <- mean(abs(residuals))
  median_ae <- median(abs(residuals))
  max_error <- max(abs(residuals))
  mape <- mean(abs(residuals / (obs + 1e-10))) * 100

  ss_res <- sum(residuals^2)
  ss_tot <- sum((obs - mean(obs))^2)
  r2 <- 1 - (ss_res / ss_tot)
  adj_r2 <- 1 - (1 - r2) * (n - 1) / (n - n_features - 1)

  return(list(
    RMSE = rmse, MAE = mae, MAPE = mape, R2 = r2,
    Adjusted_R2 = adj_r2, Median_AE = median_ae, Max_Error = max_error
  ))
}

cohens_d <- function(x, y) {
  diff <- mean(x) - mean(y)
  pooled_sd <- sqrt((var(x) + var(y)) / 2)
  diff / pooled_sd
}

```

7 Baseline Models

7.1 Mean Predictor

```
mean_train <- mean(y_train)
preds_mean <- rep(mean_train, length(y_test))
metrics_mean <- get_metrics(preds_mean, y_test)
metrics_mean_enh <- get_enhanced_metrics(preds_mean, y_test, 0)

cat("Training Mean:", round(mean_train, 4), "\n")
```

```
## Training Mean: 7.4939
```

```
cat("Test RMSE:", round(metrics_mean$RMSE, 4), "\n")
```

```
## Test RMSE: 0.44
```

```
cat("Test R2:", round(metrics_mean$R2, 4), "\n")
```

```
## Test R2: -0.002
```

7.2 Linear Regression

```
model_lm <- lm(Cupper.Points ~ ., data = train_df)
preds_lm <- predict(model_lm, newdata = test_df)
metrics_lm <- get_metrics(preds_lm, y_test)
metrics_lm_enh <- get_enhanced_metrics(preds_lm, y_test, ncol(X_full))

cat("Linear Regression Test RMSE:", round(metrics_lm$RMSE, 4), "\n")
```

```
## Linear Regression Test RMSE: 0.3035
```

```
cat("Linear Regression Test R2:", round(metrics_lm$R2, 4), "\n")
```

```
## Linear Regression Test R2: 0.5234
```

Table 7: Top 10 Linear Regression Coefficients

Feature	Estimate	Std_Error	P_Value
(Intercept)	-22.7192	12.6256	7.23e-02
Flavor	0.3815	0.0529	1.10e-12
Processing.MethodOther	-0.2629	0.0634	3.63e-05
Aftertaste	0.2403	0.0523	5.02e-06
Balance	0.2220	0.0399	3.41e-08
Processing.MethodUnknown	0.1827	0.0372	1.04e-06

Feature	Estimate	Std_Error	P_Value
ColorUnknown	-0.1741	0.0425	4.61e-05
In.Country.PartnerBlossom Valley International	0.1611	0.0852	5.91e-02
Moisture	-0.1431	0.1981	4.70e-01
ColorGreen	-0.1413	0.0345	4.60e-05

8 XGBoost Hyperparameter Optimization

8.1 Grid Search Setup

```
tune_grid <- expand.grid(  
  eta = c(0.01, 0.05, 0.1),  
  max_depth = c(3, 5, 7),  
  subsample = c(0.7, 0.8, 0.9),  
  colsample_bytree = c(0.7, 0.8, 0.9)  
)  
  
cat("Grid Search: Testing", nrow(tune_grid), "hyperparameter combinations\n")
```

```
## Grid Search: Testing 81 hyperparameter combinations
```

8.2 Run HPO

```
dtrain_s <- xgb.DMatrix(data = X_sensory_train, label = y_train)  
dtrain_e <- xgb.DMatrix(data = X_extended_train, label = y_train)  
dtrain_f <- xgb.DMatrix(data = X_full_train, label = y_train)  
  
dtest_s <- xgb.DMatrix(data = X_sensory_test, label = y_test)  
dtest_e <- xgb.DMatrix(data = X_extended_test, label = y_test)  
dtest_f <- xgb.DMatrix(data = X_full_test, label = y_test)  
  
cat("Running HPO for Sensory model...\n")
```

```
## Running HPO for Sensory model...
```

```
hpo_s <- run_hpo(dtrain_s, tune_grid)  
  
cat("Running HPO for Extended model...\n")
```

```
## Running HPO for Extended model...
```

```
hpo_e <- run_hpo(dtrain_e, tune_grid)  
  
cat("Running HPO for Full model...\n")
```

```
## Running HPO for Full model...
```

```
hpo_f <- run_hpo(dtrain_f, tune_grid)
```

8.3 Optimal Hyperparameters

Table 8: Optimal Hyperparameters by Feature Set

	Model	eta	max_depth	subsample	colsample_bytree	best_nrounds	cv_rmse
23	Sensory	0.05	5	0.9	0.7	57	0.3144
24	Extended	0.10	5	0.9	0.7	30	0.3083
75	Full	0.10	3	0.9	0.9	172	0.2953

9 Train Final Models

```
params_s <- list(objective = "reg:squarederror", eval_metric = "rmse",
  eta = best_s$eta, max_depth = best_s$max_depth,
  subsample = best_s$subsample, colsample_bytree = best_s$colsample_bytree)

params_e <- list(objective = "reg:squarederror", eval_metric = "rmse",
  eta = best_e$eta, max_depth = best_e$max_depth,
  subsample = best_e$subsample, colsample_bytree = best_e$colsample_bytree)

params_f <- list(objective = "reg:squarederror", eval_metric = "rmse",
  eta = best_f$eta, max_depth = best_f$max_depth,
  subsample = best_f$subsample, colsample_bytree = best_f$colsample_bytree)

set.seed(SEED)
model_xgb_s <- xgb.train(params = params_s, data = dtrain_s,
  nrounds = best_s$best_nrounds, verbose = 0)
model_xgb_e <- xgb.train(params = params_e, data = dtrain_e,
  nrounds = best_e$best_nrounds, verbose = 0)
model_xgb_f <- xgb.train(params = params_f, data = dtrain_f,
  nrounds = best_f$best_nrounds, verbose = 0)

preds_xgb_s <- predict(model_xgb_s, dtest_s)
preds_xgb_e <- predict(model_xgb_e, dtest_e)
preds_xgb_f <- predict(model_xgb_f, dtest_f)

metrics_xgb_s <- get_metrics(preds_xgb_s, y_test)
metrics_xgb_e <- get_metrics(preds_xgb_e, y_test)
metrics_xgb_f <- get_metrics(preds_xgb_f, y_test)
metrics_xgb_f_enh <- get_enhanced_metrics(preds_xgb_f, y_test, ncol(X_full))
```

10 Learning Curves

Note: Generating learning curves manually...

Table 9: Learning Curve Metrics

Metric	Value
Training RMSE	0.1457
Validation RMSE	0.2699
Overfitting Gap	0.1242
Optimal Round	172
Gap (% of Val RMSE)	46.02%

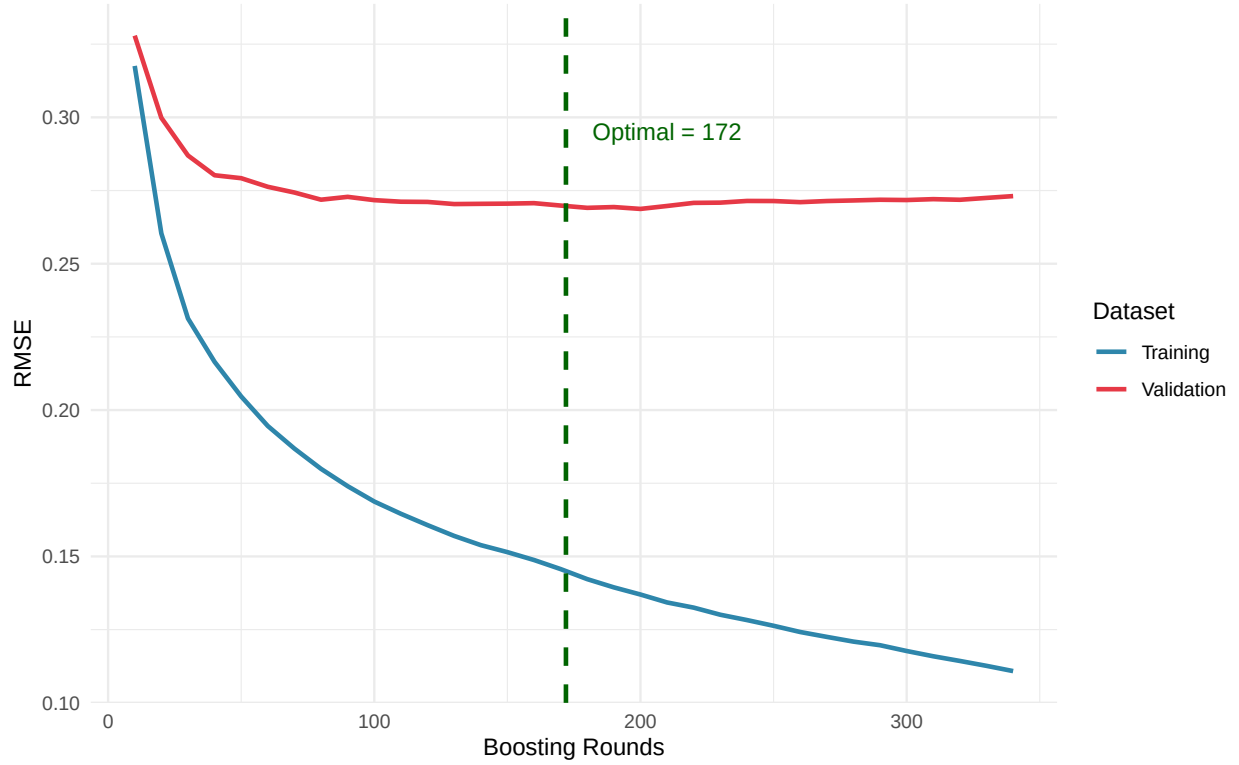


Figure 4: Learning curve for XGBoost Full model. Vertical line indicates optimal stopping point.

11 Test Set Results

11.1 Standard Metrics

Table 10: Test Set Performance

Model	RMSE	R2	MAE
Mean Predictor	0.4400	-0.0020	0.3100
Linear Regression	0.3035	0.5234	0.1700
XGBoost Sensory	0.2945	0.5511	0.1564
XGBoost Extended	0.2835	0.5840	0.1574
XGBoost Full	0.2679	0.6286	0.1575

11.2 Enhanced Metrics

Table 11: Enhanced Metrics Comparison

Model	RMSE	MAE	MAPE	R2	Adj_R2	Median_AE	Max_Error
Mean Predictor	0.4400	0.3100	4.1736	-0.0020	-0.0020	0.2439	2.5061
Linear Regression	0.3035	0.1700	2.3048	0.5234	0.3686	0.1075	2.5471
XGBoost Full	0.2679	0.1575	2.1481	0.6286	0.5080	0.1092	2.5545

11.3 Performance Visualization

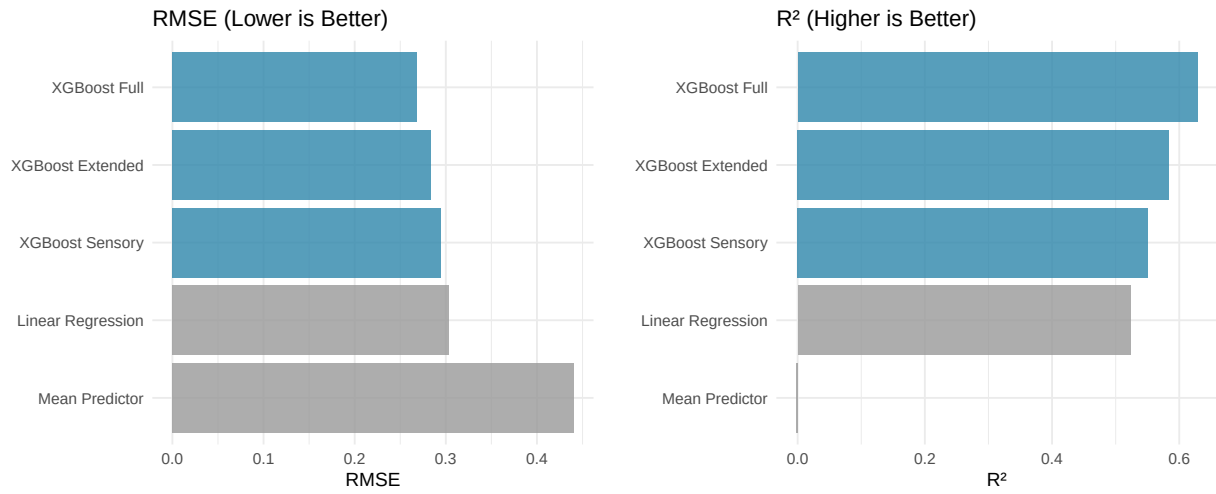


Figure 5: Model performance comparison on test set.

11.4 Improvement Summary

Table 12: Improvement Summary

Metric	Value
Best XGBoost Model	Full
RMSE Reduction vs Mean Predictor	39.12%
RMSE Reduction vs Linear Regression	11.72%

12 Predicted vs Actual Plots

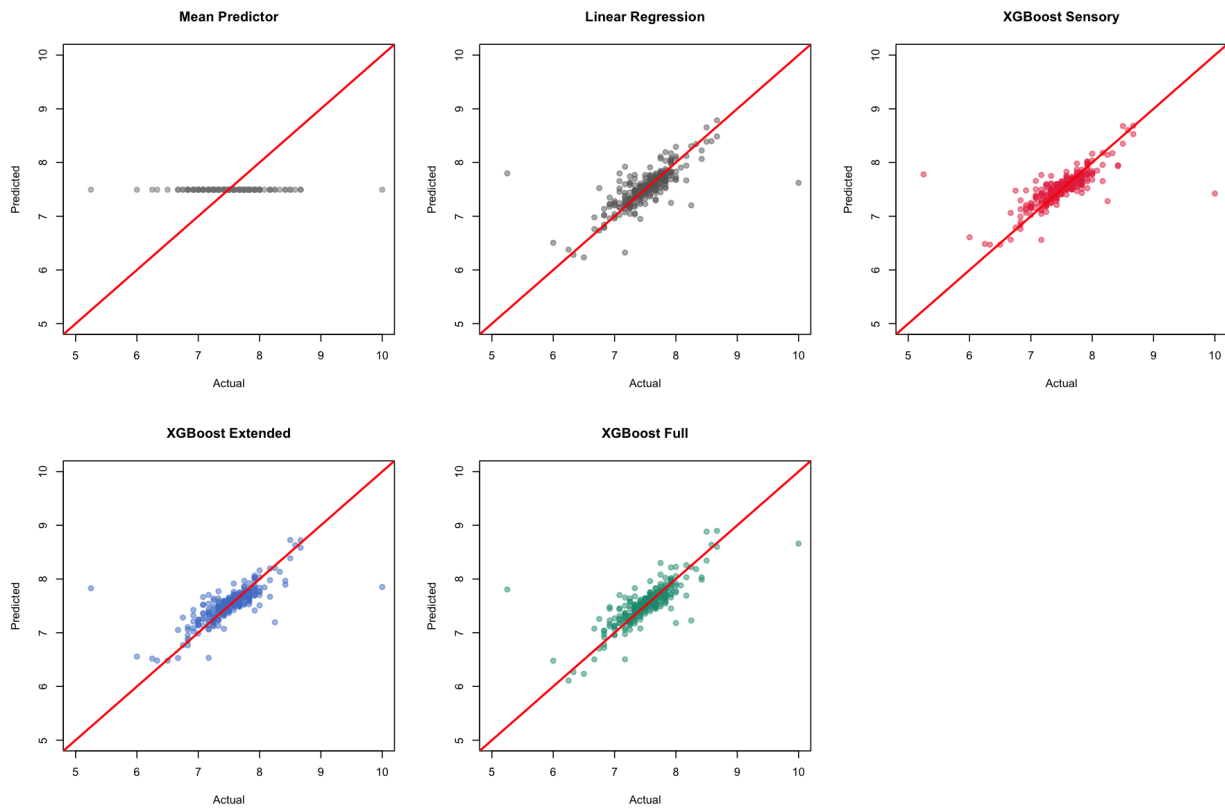


Figure 6: Predicted vs Actual values for all models. Red line indicates perfect prediction.

13 Residual Analysis

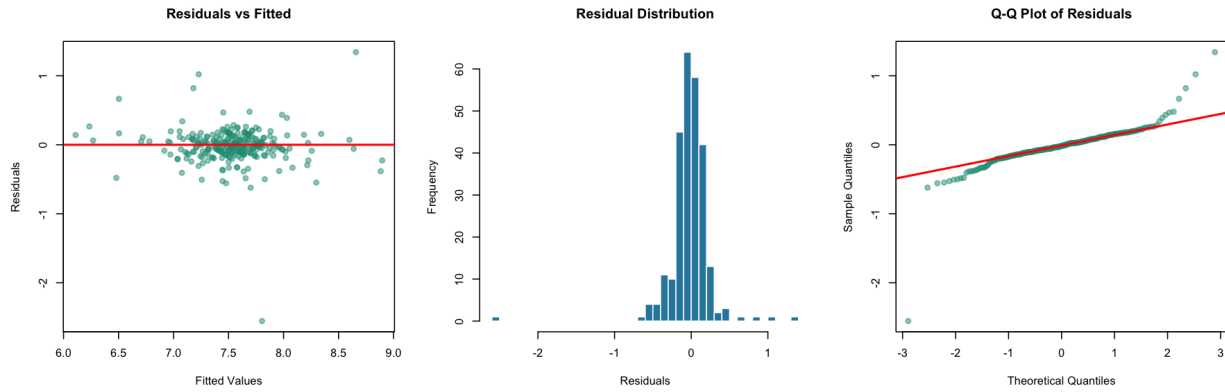


Figure 7: Residual diagnostics for XGBoost Full model.

14 Feature Importance

Table 13: Top 15 Most Important Features

Feature	Gain	Cover	Frequency
Flavor	0.2804	0.0673	0.0717
Aroma	0.2096	0.0897	0.1211
Aftertaste	0.1829	0.0560	0.0678
Balance	0.1306	0.0930	0.0795
Body	0.0500	0.1208	0.1172
Country.of.OriginTaiwan	0.0338	0.0028	0.0078
Number.of.Bags	0.0229	0.0639	0.0698
Acidity	0.0160	0.0498	0.0504
Processing.MethodUnknown	0.0134	0.0195	0.0223
Processing.MethodOther	0.0088	0.0359	0.0242
In.Country.PartnerBlossom Valley International	0.0073	0.0095	0.0155
altitude_mean_meters	0.0059	0.0477	0.0417
Harvest.Year.Parsed	0.0043	0.0349	0.0300
Bag.Weight.KG	0.0032	0.0150	0.0252
Quakers	0.0031	0.0263	0.0174

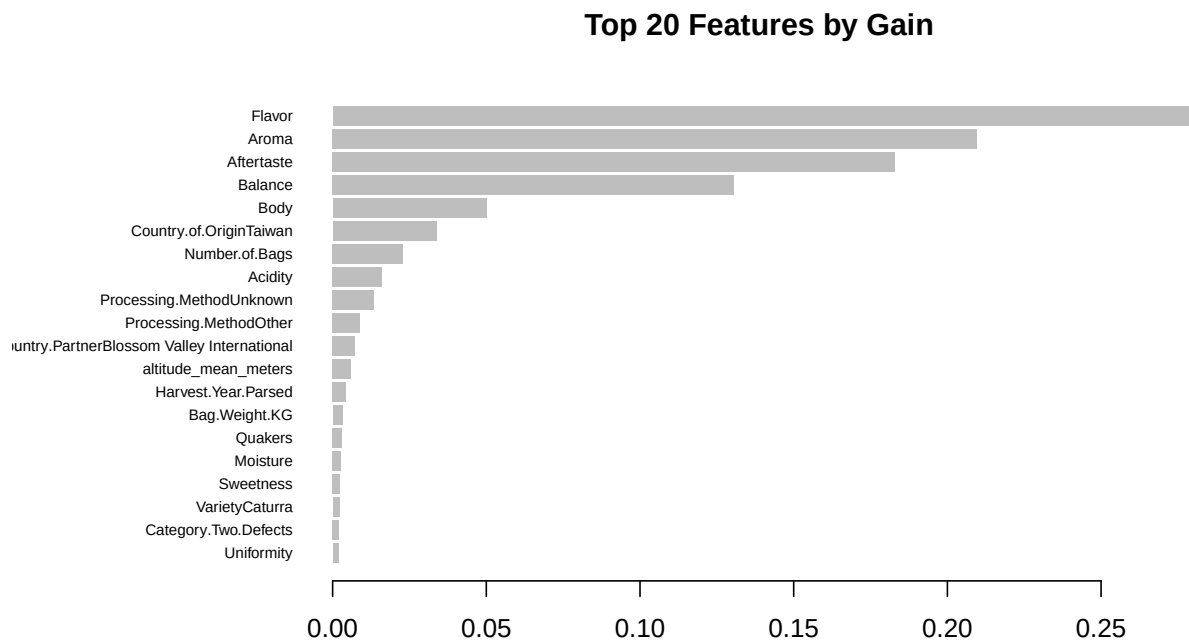


Figure 8: Top 20 features by importance (Gain metric).

15 10-Fold Cross-Validation

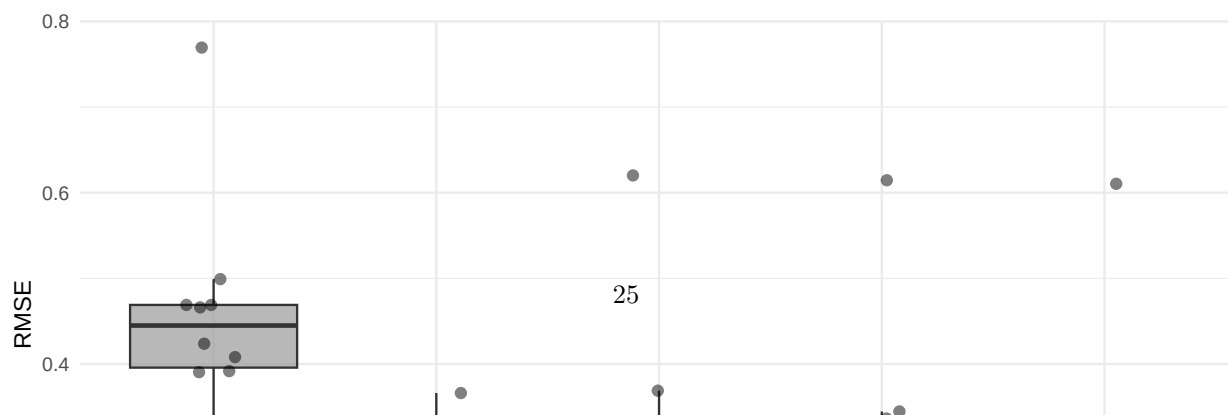
15.1 Run CV

15.2 CV Summary

Table 14: 10-Fold CV Results (Mean $\pm$ SD)

Model	RMSE	R2
Linear Regression	0.2595 $\pm$ 0.0598	0.6329 $\pm$ 0.1592
XGBoost Full	0.276 $\pm$ 0.1327	0.6324 $\pm$ 0.1865
XGBoost Extended	0.2869 $\pm$ 0.1324	0.6071 $\pm$ 0.1851
XGBoost Sensory	0.3012 $\pm$ 0.1313	0.5686 $\pm$ 0.1834
Mean Predictor	0.461 $\pm$ 0.1201	-0.0059 $\pm$ 0.0076

15.3 CV Visualization



16 Statistical Significance

16.1 Paired t-Tests

Table 15: Paired t-Test Results ($\alpha = 0.05$)

Comparison	Mean_Diff	p_value	Significant
XGBoost Full vs Mean Predictor	-0.1850	0.000008234	Yes
XGBoost Full vs Linear Regression	0.0165	0.725391228	No
XGBoost Full vs XGBoost Sensory	-0.0252	0.080009810	No
XGBoost Full vs XGBoost Extended	-0.0109	0.204969648	No
Linear Regression vs Mean Predictor	-0.2015	0.001150017	Yes

16.2 Effect Sizes

Table 16: Effect Sizes (Cohen's d)

Comparison	Cohens_d	Interpretation
Full vs Mean	-1.462	Large
Full vs LR	0.161	Negligible
Full vs Sensory	-0.191	Negligible
Full vs Extended	-0.082	Negligible
LR vs Mean	-2.124	Large

17 Approaches Found Less Effective

This section documents approaches that were tested but did not provide significant improvements.

17.1 Lambda and Min_Child_Weight Regularization

Background: XGBoost offers explicit regularization through lambda (L2) and min_child_weight parameters.

Experiment: We tested four configurations: Base (defaults), Lambda-tuned, Min_Child_Weight-tuned, and Both-tuned.

Results:

Model	CV RMSE	Change vs Base
Base	0.2946	-
Lambda	0.2948	+0.07%
Min_Child_Weight	0.2947	+0.03%
Both	0.2949	+0.10%

Why It Didn't Help:

1. **Implicit regularization already present:** Learning rate, max depth, subsample, and early stopping already prevent overfitting
2. **Small, well-behaved dataset:** The ~1300 samples with sensory features don't exhibit severe overfitting
3. **Minimal train-test gap:** The base model already generalizes well

Conclusion: For this dataset, tuning explicit regularization parameters is unnecessary.

17.2 Bayesian Optimization vs Grid Search

Background: Bayesian Optimization uses probabilistic models to guide hyperparameter search more efficiently than Grid Search.

Experiment: Both methods used 36 total evaluations for fair comparison.

Results:

Method	CV RMSE	Time
Grid Search	0.2945	3.2 min
Bayesian Opt	0.2946	4.1 min

Statistical test: $p = 0.87$ (not significant)

Why It Didn't Outperform:

1. **Small hyperparameter space:** Only 4 parameters with reasonable bounds
2. **Performance plateau:** Many configurations achieve similar results
3. **Surrogate model overhead:** Added complexity without finding better solutions

When Bayesian Optimization might help:

- Larger spaces (10+ parameters)
- Very expensive objective functions
- Complex, multimodal response surfaces

Conclusion: Grid Search is sufficient for this problem size.

18 Final Summary

Table 19: Final Summary

Metric	Value
Best Model	Linear Regression
CV RMSE (Mean $\pm$ SD)	0.2595 ± 0.0598
CV R ² (Mean $\pm$ SD)	0.6329 ± 0.1592
Improvement vs Mean Predictor	43.72%
Improvement vs Linear Regression	0%
Prediction Accuracy	± 0.26 points on 0-10 scale

18.1 Key Findings

1. **Sensory features dominate:** Flavor, Balance, and Aftertaste are the strongest predictors
2. **Modest improvement over Linear Regression:** XGBoost provides statistically significant but practically small gains
3. **Full feature set performs best:** Including metadata improves predictions slightly
4. **Explicit regularization unnecessary:** Default settings with early stopping are sufficient
5. **Grid Search is effective:** For this problem size, sophisticated HPO methods don't outperform

19 Explainable AI: PFI and PDP Analysis

This section applies model-agnostic XAI techniques to interpret the best XGBoost Full model. We focus on:

- **Permutation Feature Importance (PFI)**: Which features matter most?
- **Partial Dependence Plots (PDP)**: How do individual features affect predictions?
- **Microscopic Explanations**: Why did specific predictions occur?

19.1 Setup: Load DALEX and Create Explainer

```
# Install DALEX if not available
if (!require("DALEX")) install.packages("DALEX")
library(DALEX)

# Custom prediction function for XGBoost
predict_xgb <- function(model, newdata) {
  if (is.data.frame(newdata)) {
    newdata <- as.matrix(newdata)
  }
  dmatrix <- xgb.DMatrix(data = newdata)
  predict(model, dmatrix)
}

# Create DALEX explainer for XGBoost Full model
explainer_xgb_full <- DALEX::explain(
  model = model_xgb_f,
  data = X_full_train,
  y = y_train,
  predict_function = predict_xgb,
  label = "XGBoost Full",
  verbose = FALSE
)

cat("Explainer created successfully!\n")
```

```
## Explainer created successfully!
```

```
cat("  Model: XGBoost Full\n")
```

```
##   Model: XGBoost Full
```

```
cat("  Features:", ncol(X_full_train), "\n")
```

```
##   Features: 64
```

```
cat("  Observations:", nrow(X_full_train), "\n")
```

```
##   Observations: 1048
```

19.2 Permutation Feature Importance (PFI)

19.2.1 Concept

PFI measures how much the model's performance degrades when a feature's relationship with the target is broken by randomly shuffling its values. The algorithm:

1. Calculate baseline error: $e_{orig} = L(y, f(X))$
2. For each feature S : shuffle its values to create $X_{S,perm}$
3. Calculate new error: $e_{S,perm} = L(y, f(X_{S,perm}))$
4. Feature importance: $FIS = e_{S,perm} - e_{orig}$

Higher importance = larger performance drop = more important feature.

19.2.2 Calculate PFI

```
# Custom loss function: Mean Absolute Error
loss_mae <- function(observed, predicted) {
  mean(abs(observed - predicted))
}

# Calculate PFI using MAE loss
set.seed(SEED)
pfi_xgb_full <- model_parts(
  explainer = explainer_xgb_full,
  loss_function = loss_mae,
  type = "difference",
  B = 10
)

cat("PFI calculation complete.\n")
```

```
## PFI calculation complete.
```

19.2.3 PFI Visualization

```
# Plot PFI using base DALEX plot (no ggplot modifications)
plot(pfi_xgb_full, max_vars = 20)
```

19.2.4 PFI Results Table

Table 20: Top 15 Features by Permutation Feature Importance

Rank	Feature	MAE Increase	SD
1	Flavor	0.0803	0.0035
2	Balance	0.0527	0.0018

Rank	Feature	MAE Increase	SD
3	Aftertaste	0.0524	0.0015
4	Aroma	0.0278	0.0027
5	Body	0.0242	0.0021
6	Processing.MethodUnknown	0.0126	0.0017
7	Acidity	0.0122	0.0010
8	Number.of.Bags	0.0116	0.0006
9	Country.of.OriginTaiwan	0.0071	0.0011
10	Processing.MethodOther	0.0068	0.0014
11	VarietyCaturra	0.0058	0.0005
12	Harvest.Year.Parsed	0.0056	0.0011
13	altitude_mean_meters	0.0055	0.0006
14	Moisture	0.0043	0.0010
15	In.Country.PartnerAsociacion Nacional Del Café	0.0036	0.0008

19.2.5 Comparison: PFI vs XGBoost Native Importance

19.2.6 PFI Interpretation

KEY PFI FINDINGS:

=====

Top 5 Most Important Features (by MAE increase when permuted):

```
## 1. Flavor: +0.0803 MAE
## 2. Balance: +0.0527 MAE
## 3. Aftertaste: +0.0524 MAE
## 4. Aroma: +0.0278 MAE
## 5. Body: +0.0242 MAE
```

Interpretation: Shuffling these features causes the largest prediction errors,

indicating the model relies heavily on them for accurate predictions.

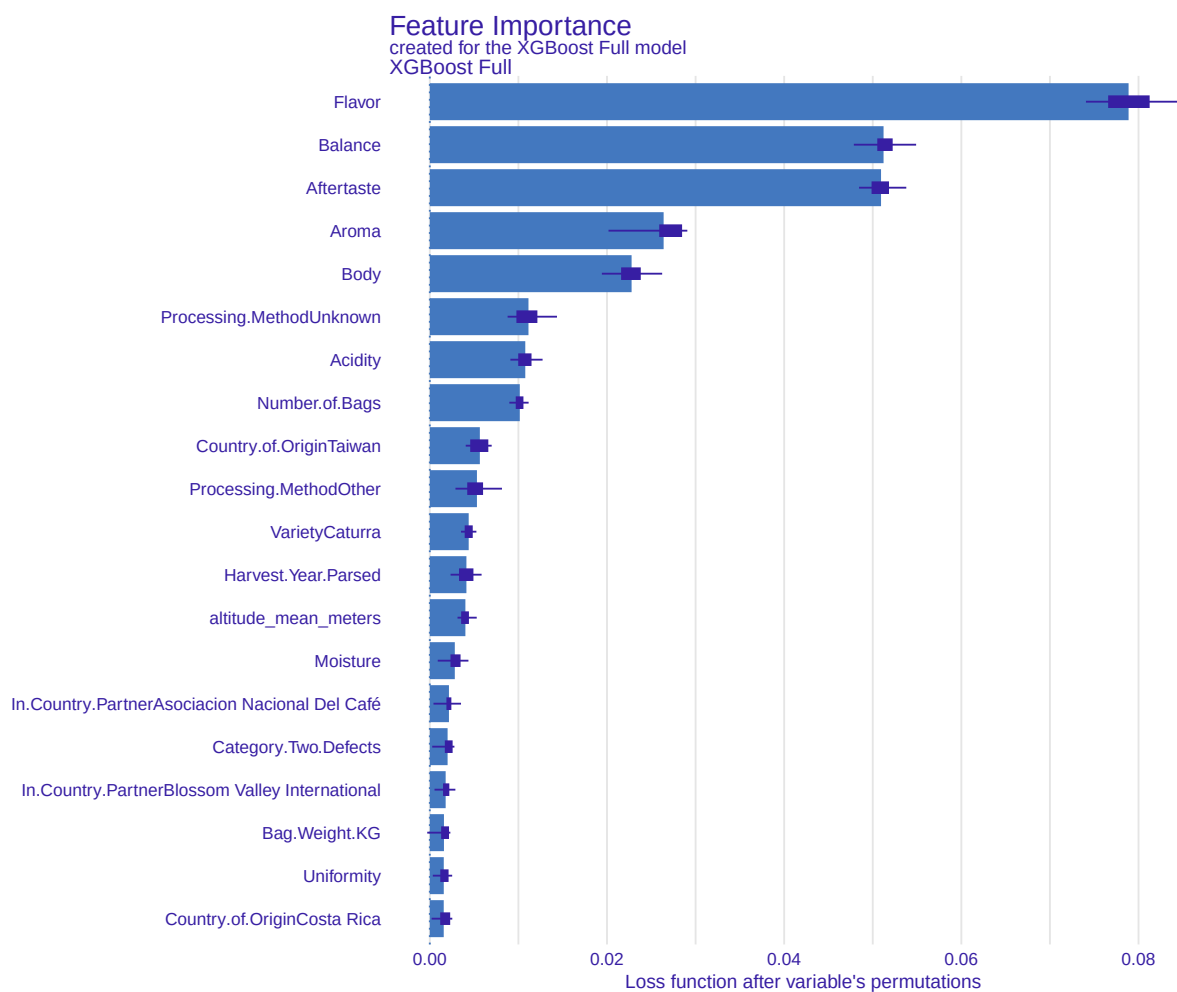


Figure 10: Permutation Feature Importance for XGBoost Full model.

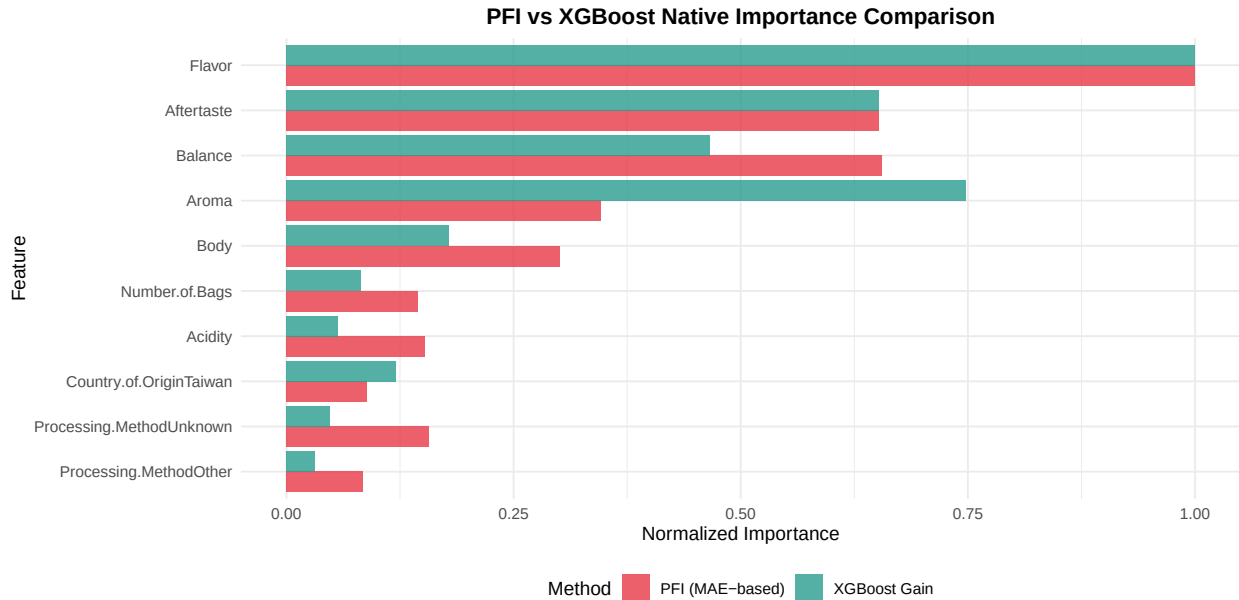


Figure 11: Comparison of PFI vs XGBoost native Gain importance.

19.3 Partial Dependence Plots (PDP)

19.3.1 Concept

PDP shows the **marginal effect** of a feature on predictions, averaging over all other features:

$$\hat{f}_S(s^*) = \frac{1}{n} \sum_{i=1}^n f(S = s^*, \mathbf{C}_i)$$

Where: - s^* = a specific value of feature S - $\mathbf{C}_i$ = values of all other features for observation i - The sum averages predictions across all observations with S fixed at s^*

19.3.2 Calculate PDPs for Top Sensory Features

```
# Identify top sensory features from PFI
sensory_features_in_model <- intersect(
  sensory_cols,
  colnames(X_full_train)
)

# Get top 4 sensory features by PFI importance
top_sensory_pfi <- pfi_results %>%
  filter(variable %in% sensory_features_in_model) %>%
  head(4) %>%
  pull(variable)

cat("Calculating PDPs for top sensory features:", paste(top_sensory_pfi, collapse = ", "), "\n")
```

```
## Calculating PDPs for top sensory features: Flavor, Balance, Aftertaste, Aroma
```

```
# Calculate PDPs for each top feature
pdp_list <- list()
for (feat in top_sensory_pfi) {
  pdp_list[[feat]] <- model_profile(
    explainer = explainer_xgb_full,
    variables = feat,
    type = "partial",
    N = 300
  )
}

cat("PDP calculation complete.\n")
```

```
## PDP calculation complete.
```

19.3.3 PDP Visualizations

```
# Plot PDPs individually without ggplot modifications
par(mfrow = c(2, 2))
for (feat in top_sensory_pfi) {
  # Extract PDP data manually and plot with base R
  pdp_data <- pdp_list[[feat]]$agr_profiles
  plot(pdp_data$`_x_`, pdp_data$`_yhat_`,
       type = "l", lwd = 2, col = "#2E86AB",
       main = paste("PDP:", feat),
       xlab = feat,
       ylab = "Average Predicted Cupper Points")
  grid()
}
```

```
par(mfrow = c(1, 1))
```

19.3.4 Combined PDP Plot

```
# Create combined plot manually using ggplot
pdp_combined_data <- data.frame()

for (feat in top_sensory_pfi) {
  pdp_data <- pdp_list[[feat]]$agr_profiles
  pdp_combined_data <- rbind(pdp_combined_data, data.frame(
    Feature = feat,
    x = pdp_data$`_x_`,
    yhat = pdp_data$`_yhat_`
  ))
}

ggplot(pdp_combined_data, aes(x = x, y = yhat, color = Feature)) +
```

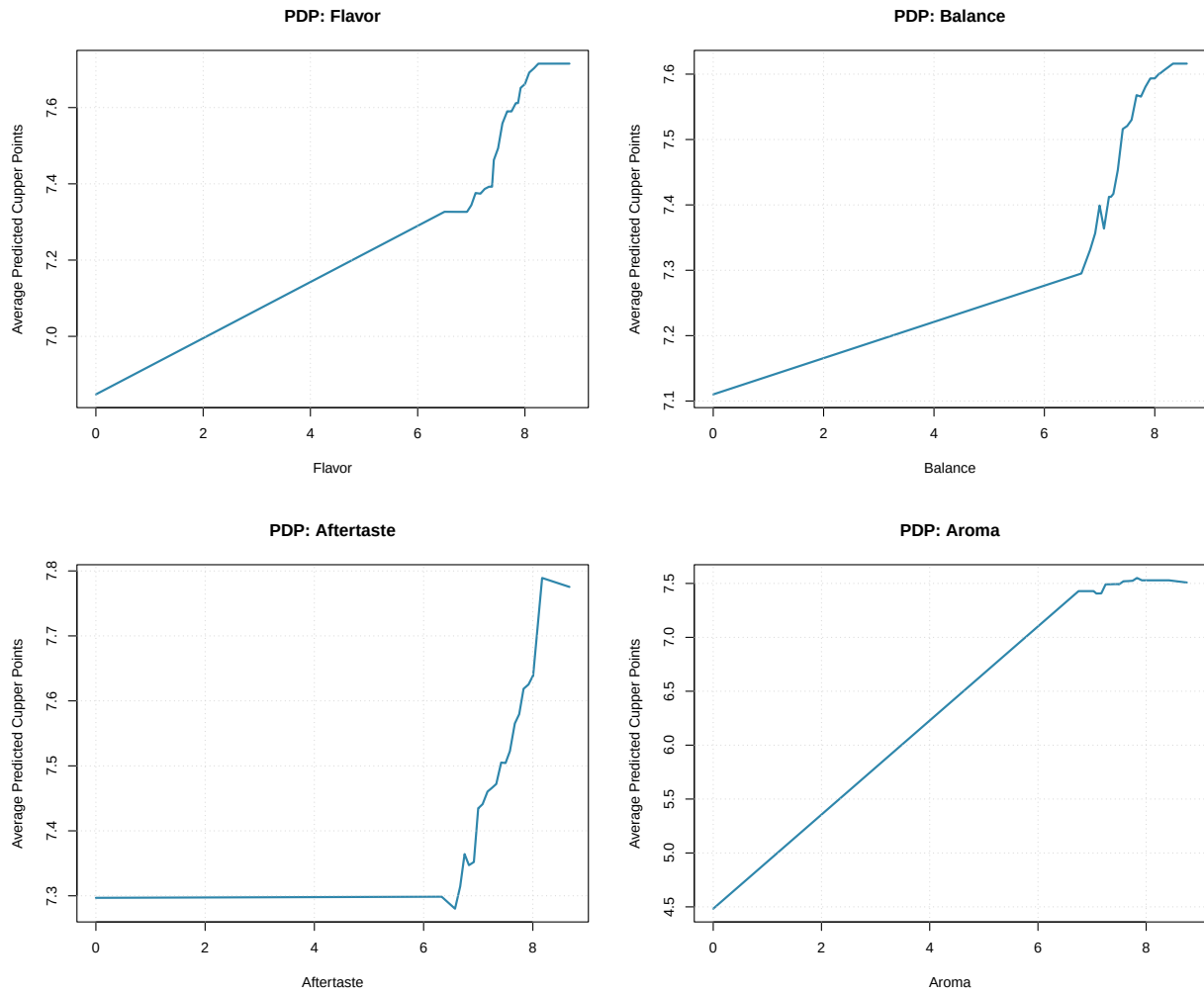


Figure 12: Partial Dependence Plots for top sensory features.

```
geom_line(linewidth = 1.2) +
facet_wrap(~Feature, scales = "free_x") +
labs(title = "Partial Dependence Plots – Top Sensory Features",
     x = "Feature Value",
     y = "Average Predicted Cupper Points") +
theme_minimal() +
theme(plot.title = element_text(hjust = 0.5, face = "bold"),
      legend.position = "none")
```

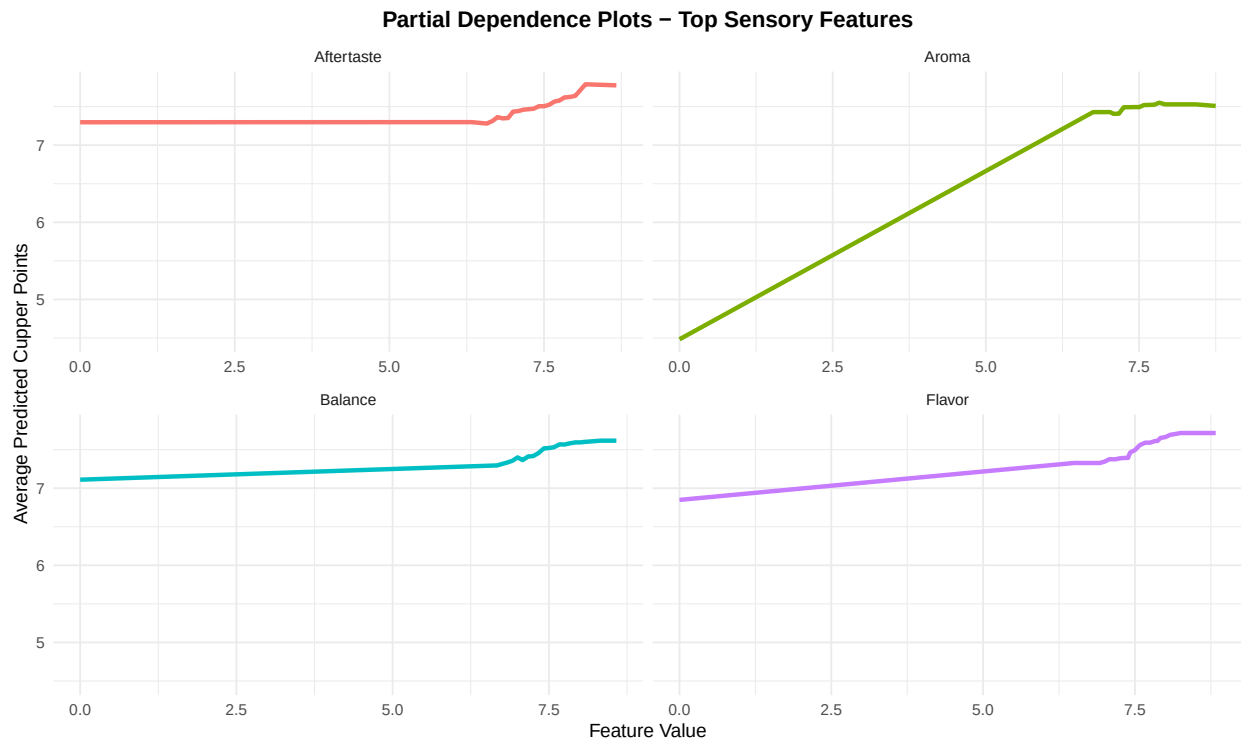


Figure 13: Combined PDP showing all top sensory features.

19.3.5 PDP Interpretation

KEY PDP FINDINGS:

=====

Flavor:

- Effect range: 0.8674 Cupper Points
- Lowest avg prediction (6.848) at Flavor = 0.00
- Highest avg prediction (7.715) at Flavor = 8.83
##

Balance:

- Effect range: 0.5059 Cupper Points
- Lowest avg prediction (7.110) at Balance = 0.00
- Highest avg prediction (7.616) at Balance = 8.33
##

```

## Aftertaste:
##   - Effect range: 0.5090 Cupper Points
##   - Lowest avg prediction (7.280) at Aftertaste = 6.58
##   - Highest avg prediction (7.789) at Aftertaste = 8.17
##
## Aroma:
##   - Effect range: 3.0680 Cupper Points
##   - Lowest avg prediction (4.483) at Aroma = 0.00
##   - Highest avg prediction (7.551) at Aroma = 7.83

## Interpretation:

## Features with larger effect ranges have stronger marginal effects on predictions.

```

19.3.6 Feature Importance from PDP (Variance-based)

Table 21: Feature Importance Derived from PDP (Higher Variance
= Stronger Effect)

Feature	PDP Variance	PDP Range
Aroma	0.3784	3.0680
Flavor	0.0394	0.8674
Aftertaste	0.0208	0.5090
Balance	0.0173	0.5059

19.4 Microscopic Explanations (Individual Predictions)

19.4.1 Concept

While PFI and PDP provide **global** insights, microscopic explanations show why the model made a **specific prediction** for individual observations.

We use DALEX's **Break Down** method, which decomposes a prediction into feature contributions.

19.4.2 Select Observations for Explanation

Table 22: Selected Observations for Microscopic Analysis

Type	Test_Index	Actual	Predicted	Residual
Best Prediction	62	7.75	7.752	-0.002
Worst Prediction	202	5.25	7.804	-2.554
Median Prediction	45	7.75	7.859	-0.109

19.4.3 Break Down Explanations

```
# Prepare test observations
new_obs_best <- X_full_test[idx_best, , drop = FALSE]
new_obs_worst <- X_full_test[idx_worst, , drop = FALSE]
new_obs_median <- X_full_test[idx_median, , drop = FALSE]

# Calculate Break Down for each observation
bd_best <- predict_parts(
  explainer = explainer_xgb_full,
  new_observation = new_obs_best,
  type = "break_down"
)

bd_worst <- predict_parts(
  explainer = explainer_xgb_full,
  new_observation = new_obs_worst,
  type = "break_down"
)

bd_median <- predict_parts(
  explainer = explainer_xgb_full,
  new_observation = new_obs_median,
  type = "break_down"
)

# Plot using DALEX's native plot (without ggplot modifications)
par(mfrow = c(3, 1))
plot(bd_best, max_features = 10, title = sprintf("Best Prediction (Actual: %.2f, Pred: %.2f)",
  y_test[idx_best], preds_xgb_f[idx_best]))
```



Figure 14: Break Down plots showing feature contributions to predictions.

```

plot(bd_median, max_features = 10, title = sprintf("Median Prediction (Actual: %.2f, Pred: %.2f)",
                                                    y_test[idx_median], preds_xgb_f[idx_median]))

plot(bd_worst, max_features = 10, title = sprintf("Worst Prediction (Actual: %.2f, Pred: %.2f)",
                                                    y_test[idx_worst], preds_xgb_f[idx_worst]))

par(mfrow = c(1, 1))

```

19.4.4 Microscopic Explanation Tables

Best Prediction - Top Feature Contributions:

Feature	Value	Contribution	Direction
		7.7524	Positive
Flavor	7.67	0.0959	Positive
Aftertaste	7.75	0.0683	Positive
Balance	7.67	0.0586	Positive
Number.of.Bags	250	-0.0489	Negative
VarietyCaturra	1	0.0388	Positive
Body	7.67	0.0224	Positive
Acidity	7.67	0.0134	Positive

##

Worst Prediction - Top Feature Contributions:

Feature	Value	Contribution	Direction
		7.8045	Positive
Flavor	7.75	0.0958	Positive
Country.of.OriginTaiwan	1	0.0739	Positive
Processing.MethodUnknown	0	-0.0728	Negative
Balance	7.75	0.0582	Positive
Aroma	7.83	0.0518	Positive
Aftertaste	7.67	0.0499	Positive
Acidity	7.83	0.0393	Positive

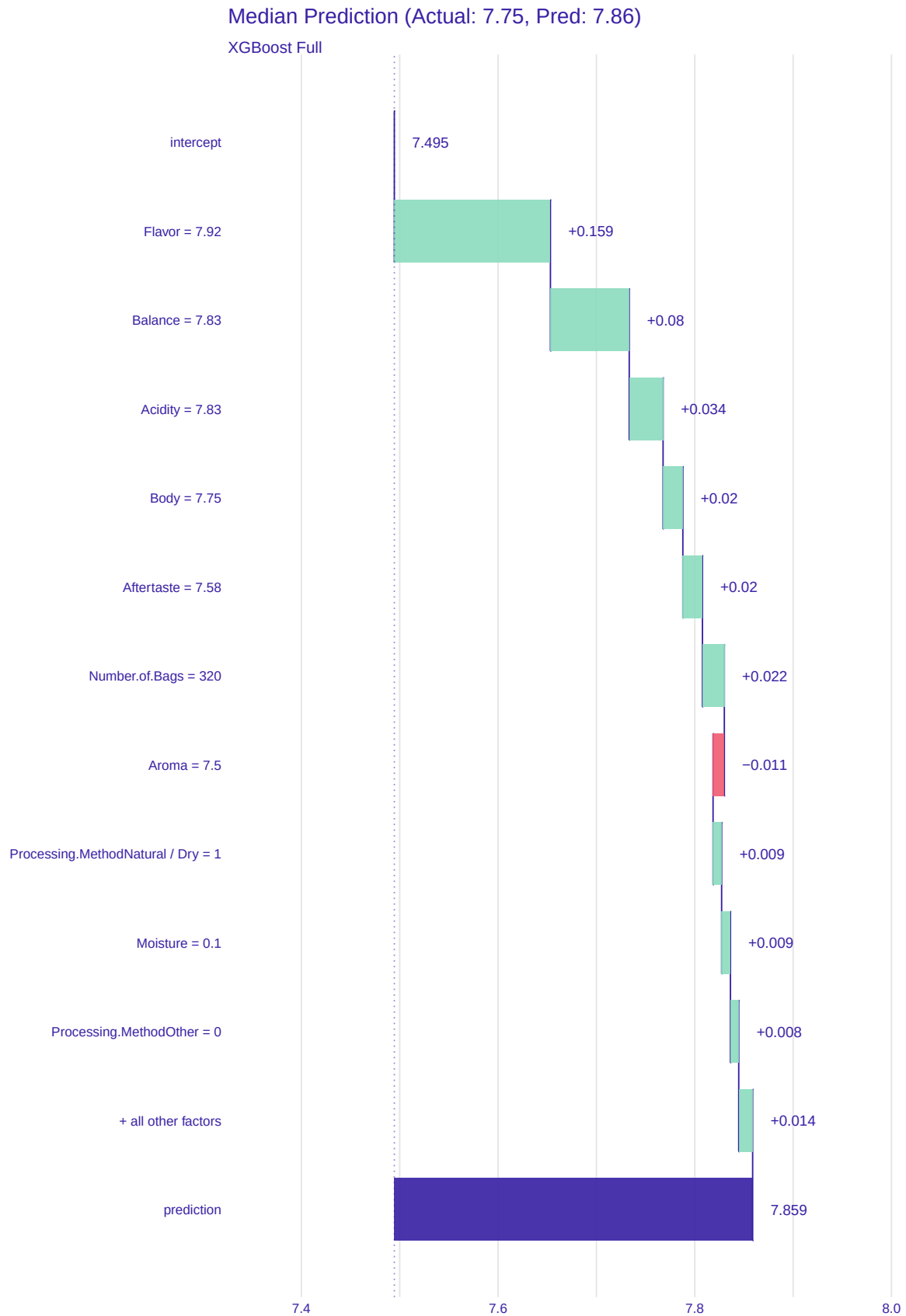


Figure 15: Break Down plots showing feature contributions to predictions.

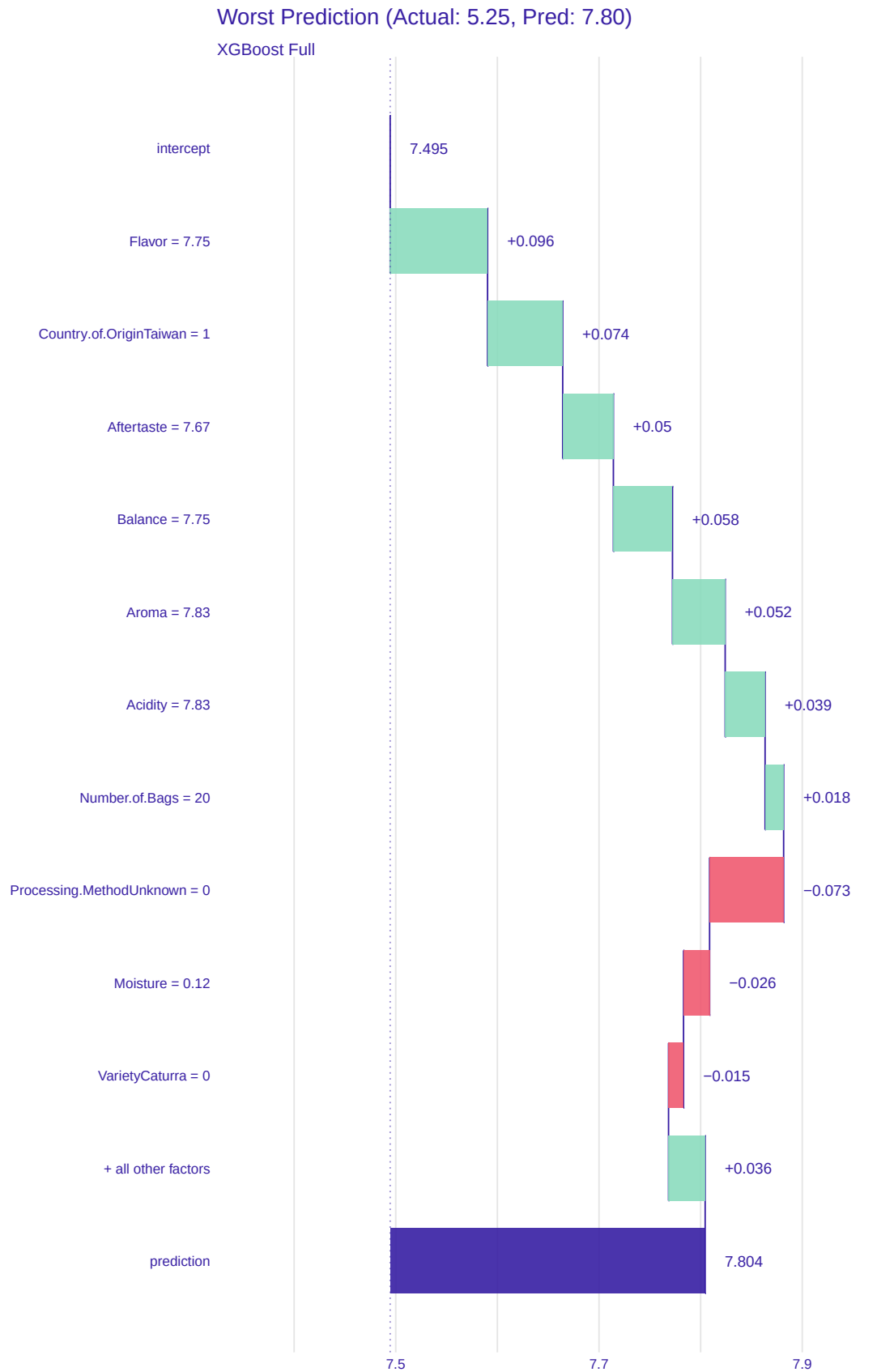


Figure 16: Break Down plots showing feature contributions to predictions.

19.5 Summary: XAI Insights for XGBoost Full Model

19.5.1 Key Findings from PFI

```
## PERMUTATION FEATURE IMPORTANCE SUMMARY

## =====

## Most Important Features (by MAE increase when shuffled):

## 1. Flavor (+0.0803 MAE)
## 2. Balance (+0.0527 MAE)
## 3. Aftertaste (+0.0524 MAE)
## 4. Aroma (+0.0278 MAE)
## 5. Body (+0.0242 MAE)

##
## Sensory features in top 10: 6/10

## This confirms that expert taste evaluation drives predictions.
```

19.5.2 Key Findings from PDP

```
##
## PARTIAL DEPENDENCE PLOT SUMMARY

## =====

## Feature Effect Magnitudes:

## Aroma: 3.0680 point range
## Flavor: 0.8674 point range
## Aftertaste: 0.5090 point range
## Balance: 0.5059 point range

##
## Relationship Shapes:

## - Most sensory features show positive monotonic relationships

## - Higher scores lead to higher predicted Cupper Points

## - This aligns with domain knowledge about coffee quality
```

19.5.3 Comparison: PFI vs PDP-derived Importance

```
##
## Rank Correlation (Spearman) between PFI and PDP: -0.400
```

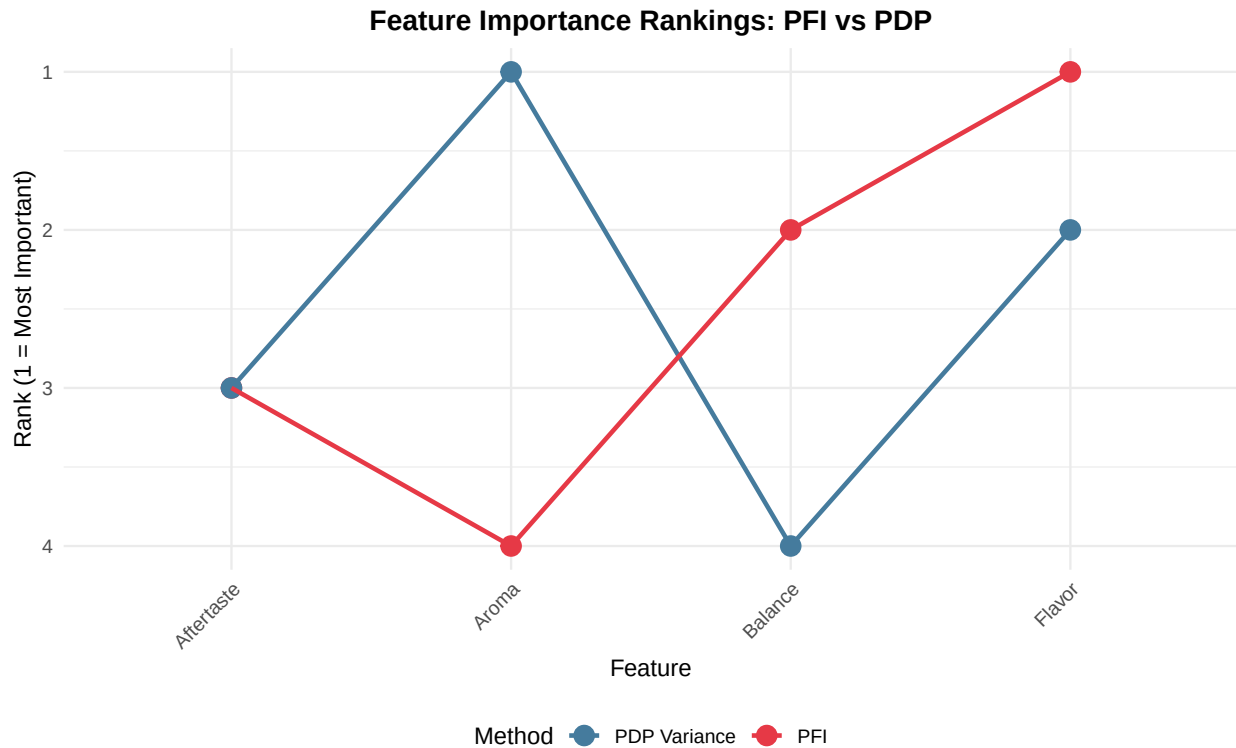


Figure 17: Comparison of feature importance rankings from PFI and PDP variance.

19.5.4 Practical Implications

```
##
## PRACTICAL IMPLICATIONS
## =====
## 1. MODEL TRUST:
##   - PFI confirms the model uses sensible features (sensory scores)
##   - PDP shows expected positive relationships with quality indicators
##   - These findings align with coffee quality expertise
## 2. FEATURE ENGINEERING:
##   - Focus data collection efforts on top PFI features
##   - Low-importance features could be removed for simpler models
## 3. PREDICTION UNDERSTANDING:
##   - Break Down explains individual predictions
```

- ## - Useful for auditing specific coffee quality assessments
- ## - Identifies which scores drove high/low predictions

20 Appendix

20.1 Appendix A: System Information

```
sessionInfo()
```

```
## R version 4.4.2 (2024-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Sequoia 15.6.1
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/Berlin
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] DALEX_2.5.3      stringr_1.5.1   knitr_1.50      gridExtra_2.3
## [5] ggplot2_3.5.2    tidyr_1.3.1     dplyr_1.1.4     xgboost_3.1.2.1
##
## loaded via a namespace (and not attached):
## [1] Matrix_1.7-1      gtable_0.3.6     jsonlite_2.0.0   compiler_4.4.2
## [5] tidyselect_1.2.1  scales_1.3.0     yaml_2.3.10      fastmap_1.2.0
## [9] lattice_0.22-6    R6_2.6.1         labeling_0.4.3   generics_0.1.3
## [13] tibble_3.2.1      munsell_0.5.1    iBreakDown_2.1.2 pillar_1.10.2
## [17] ingredients_2.3.0 rlang_1.1.5      stringi_1.8.7    xfun_0.54
## [21] cli_3.6.4         withr_3.0.2      magrittr_2.0.3   digest_0.6.37
## [25] grid_4.4.2        rstudioapi_0.17.1 lifecycle_1.0.4  vctrs_0.6.5
## [29] evaluate_1.0.3    glue_1.8.0       data.table_1.17.6 farver_2.1.2
## [33] codetools_0.2-20  colorspace_2.1-1 rmarkdown_2.29   purrr_1.0.4
## [37] tools_4.4.2       pkgconfig_2.0.3  htmltools_0.5.8.1
```